Measuring LLM Security Verdicts

Author Name
Independent Researcher — author@example.org

***Abstract*—**Large language models (LLMs) are increasingly used as *verifiers* of security findings: a scanner proposes a candidate, and a model adjudicates whether it is real. Whether such a verdict can be trusted is rarely measured. We adapt the *four-way decomposition* — an instrument previously used to show that static analyzers flag constructs rather than vulnerabilities — to LLM verifiers, and add evidence gating, an adversarial comment attack, a composition study, and a calibration analysis. Across five model families and a real-fix corpus we find that (i) an evidence-grounded LLM tracks the fix markedly more often than a pattern matcher (19–24% vs. 4.8%, no over-reporting), but (ii) it abstains on about half of all judgements and misses most genuine fixes, so its bottleneck is *coverage*, not accuracy; (iii) almost every successful adversarial evasion is an *abstention* rather than a false positive, and an adaptive attacker reaches 50% evasion where fixed templates reach 26%; and (iv) composing an LLM with a weaker deterministic verifier *relocates* trust — grounding a claim helps for free, while requiring consensus with a weaker tool loses recall for no gain. Our evidence is directional (a 21-pair security subset and 5-pair cross-family sets), and we state this plainly. We release a reproducible harness and argue that abstention should be measured and reported as a first-class failure mode of security agents.

***Index Terms*—**security verification, large language models, empirical measurement, abstention, adversarial robustness, static analysis, calibration.

I. Introduction

Automated vulnerability analysis has converged on a standard shape: a policy queries tools and a code graph, and is trained or prompted to report findings [1], [2], [4], [6]. As models take on the *verification* role — deciding whether a candidate is real — the question becomes whether their verdicts can be trusted. Prior work shows that LLM security *detectors* are highly susceptible to adversarial source comments [5]; what has not been measured systematically is whether an LLM *verdict* tracks the vulnerability, how it fails when it does not, and how it behaves under composition and an adversary.

A separate line of work established that benchmark scores do not predict real detection: for a static analyzer, the fraction of commits whose verdict moves with a fix can be two orders of magnitude below its benchmark true-positive rate, and the *four-way decomposition* — whether a tool flagged both revisions, the vulnerable revision only, the fixed revision only, or neither — made that visible [15]. We apply that instrument to LLM verifiers and extend it with evidence gating, an adversarial comment attack, a composition study, and calibration.

Contributions.

- **An instrument.** We apply the four-way decomposition to LLM verifiers, gate every verdict on machine-checkable evidence, and make coverage (languages checked vs. unchecked) explicit. The harness is released.
- **A coverage-bound finding.** An evidence-grounded LLM verifier tracks the fix far more often than a pattern matcher (19–24% vs. 4.8%) and does not over-report (1.0–1.25× vs. 6.0×), but abstains on ~50% of judgements and misses most genuine fixes: its bottleneck is silence, not error.
- **Abstention as the attack surface.** Under adversarial comments, almost every successful evasion is an abstention rather than a false positive; an adaptive attacker reaches 50% evasion against 26% for fixed templates.
- **Composition relocates trust.** Evidence grounding removes an ungrounded positive at no recall cost; requiring agreement with a weaker deterministic verifier loses recall and gains nothing.
- **Calibration and cost.** The verifier is well calibrated (ECE 0.025) and precise (0% fixed-side false positives) when it commits; disabling model thinking cuts cost ~8× at a measured precision cost, and a content-addressed verdict cache removes 100% of repeat cost.

We are explicit that our evidence is directional and small (Section VII); we treat it as a measurement-methodology contribution, not a claim of a large effect.

II. Related Work

LLM agents for vulnerability analysis. OpenAnt [4], FuzzingBrain V2 [7], DREA [8], CLEAR [9], and ATLANTIS [10] combine LLMs with static analysis, symbolic execution, and fuzzing. They report improved discovery; they generally do not measure whether a *verdict* tracks the vulnerability.

RL from verifiable rewards. VulAgentRL [1] uses a code property graph as both a tool and a reward verifier with exact node matching; Antares [2] trains small models with verifiable rewards; EAVA [3] couples supervised tuning with reinforcement learning. We do not train; we ask what trust such a grounded verdict earns at inference time.

Adversarial input. ALIBI [5] shows adversarial source comments defeat LLM detectors with >90% success; CoER [11] co-evolves attacker and defender for indirect prompt injection. We attack the *verifier* rather than the detector, split evasion by outcome, and include an adaptive attacker.

Abstention and calibration. AWA-RL [12] learns to abstain rather than fabricate in open-domain QA. We bring abstention into security verification and show it is the dominant adversarial target.

Benchmarks and validity. The OWASP Benchmark [13] is the standard scanner yardstick; JavaVFC [14] is a corpus of vulnerability-fixing commits. A companion study [15] showed the OWASP score is not well defined and does not predict real detection, and introduced the four-way decomposition; a 2026 survey [16] identifies weak oracles, temporal leakage, and unreported budgets as recurring validity

threats. Our work extends [15] from static tools to LLM verifiers.

III. Method

A. Four-way decomposition

For a vulnerable/fixed revision pair, we classify a verifier's verdict into one of four cells: *both* (flagged before and after the fix — not detection), *vuln-only* (flagged only before — a flip, i.e. detection), *fixed-only* (noise), and *neither* (a miss). We report the *flip rate*, the *naive coverage*, and their ratio, the *over-report factor*.

B. Verifier arms

V1 (LLM, ungated): the verdict is taken as-is. **V1g** (LLM, gated): every cited evidence reference must resolve to a real syntax node, or a positive verdict is downgraded to *unknown*. **V2** (tool): the pinned analyzer's verdict. **V5** (hybrid): flagged only if the gated LLM *and* the tool agree.

C. Bounded verdict request and evidence gating

The model never sees a repository. For a candidate it receives the candidate (rule, location, CWE hint), the enclosing function extracted from a tree-sitter code property graph, and optional retrieval. It must respond with strict JSON, cite `path:line` references, and may answer `unknown_risk`. Each reference is resolved against the graph; an unverifiable positive is downgraded.

D. Adversary

Templates: five comment strategies (authority, sanitizer, tool-output, test-passed, reviewed) inserted just above the suspected sink. *Adaptive:* an attacker model writes a comment, observes the verifier's verdict, and refines over up to three rounds [5]. We split evasion into false-positive and abstention outcomes.

E. Calibration and cost

Per revision judgement we record confidence, correctness, and disposition; we compute expected calibration error (ECE), reliability bins, a selective-risk/coverage curve, false-negative rate on vulnerable revisions, and false-positive rate on fixed revisions. Cost is reported in tokens and, at known prices, dollars per true positive.

F. Whole-codebase view

Files are mapped to a language registry with an explicit coverage label (`covered`/`find_only`/`ground_only`/`unsupported`), and a lightweight system graph connects functions, HTTP routes, configuration sources, and dependencies to produce entry-to-sink path candidates whose coverage is that of their least-covered hop.

IV. Experimental Setup

Corpora. (i) 21 Java security-fix pairs from JavaVFC [14] (784 commits; 327 usable single-file; 207 materialised offline; the strong-security subset used here is 21). (ii) 5 hand-built Python vulnerable/fixed pairs and 3 canaries. (iii) The prior study's 21-file Java control set (18 vulnerable, 3 safe). Each revision is an isolated file; the commit message is supplied as candidate context.

Analyzers. opengrep 1.30.0 with the prior study's pinned 690-rule Java ruleset (sha256 77df32e0…) and a minimal 5-rule Python ruleset.

Models. Five families via an OpenAI-compatible API: `gpt-oss-120b`, `gpt-oss-20b`, `qwen3.8-27b`, `deepseek-flash`, `deepseek-v4-pro`. The scaled JavaVFC run uses `deepseek-flash`; cross-family checks use the 5-pair Python set.

Reproducibility. All assets are pinned by hash; the control reproduction and the Python-pair measurement run fully offline; 112 tests accompany the harness.

V. Results

A. Verifier dependence

On identical inputs, models disagree in both flip rate and failure mode (Table I). The result is verifier-dependent — the phenomenon the instrument is designed to expose.

TABLE I. Cross-family verdicts on 5 Python pairs / 3 canaries.

Model	flip rate	both	canary FP
gpt-oss-120b	80%	1	1/3
gpt-oss-20b	100%	0	1/3
qwen3.8-27b	100%	0	0/3
deepseek-flash	100%	0	0/3
deepseek-v4-pro	80%	0	0/3 (1 abstain)

B. LLM versus pattern matcher, and composition

Table II reports the four arms on the 21-pair JavaVFC security subset. The tool over-reports 6.0× (it flags 5 pairs on both revisions). The LLM tracks the fix ~4× more often and barely over-reports. Evidence gating (V1g) removes the single ungrounded positive at no recall cost. Requiring agreement with the tool (V5) *loses* a flip and gains nothing.

TABLE II. Verifier arms on 21 JavaVFC security pairs.

Arm	both	vuln	fixed	neither	flip	over-rep.
V2 tool	5	1	0	15	4.8%	6.0×
V1 ungated	1	4	0	16	19.0%	1.25×
V1g gated	0	4	1	16	19.0%	1.00×
V5 hybrid	0	3	0	18	14.3%	1.00×

C. Adversarial comments

Table III reports template and adaptive evasion. The *sanitizer* tactic (“the input is validated upstream”) was strongest across families. All successful adaptive evasions, and 8 of 9 template evasions, were abstentions rather than false positives: an attacker need only make the verifier unsure.

TABLE III. Evasion. JavaVFC templates: baseline 7/21 TPs; adaptive: baseline 6/21, attacker deepseek-v4-pro vs. judge deepseek-flash.

Attack	evasion	via FP	via abstain
templates (authority)	43% (3/7)	0	3
templates (overall)	26%	1	8
adaptive (3 rounds)	50% (3/6)	0	3

D. Calibration and coverage

Table IV shows the verifier is well calibrated and precise when it commits, but silent half the time. At confidence ≥0.9, coverage falls to 43% with risk 0.11. The limiting factor is abstention, not deception.

TABLE IV. CALIBRATION ON 21 PAIRS (42 REVISION JUDGEMENTS), DEEPSEEK-FLASH.

Metric	Value
abstention rate	50.0%
detection (recall on vuln revision)	38.1%
false-negative rate	61.9%
fixed-side false-positive rate	0.0%
accuracy (scored)	81.0%
ECE	0.025

E. Cost

Output tokens dominate (~91% of cost). Disabling model thinking reduced output from 121,596 to 5,271 tokens over 42 judgements (~8× lower cost) but introduced fixed-side false positives (0% → 9.5%) and worsened ECE (0.153 → 0.207). A content-addressed verdict cache keyed on the model, prompts, and slice removed 100% of repeat cost. At off-peak prices the gated verifier costs ≈$0.0017 per judgement.

VI. DISCUSSION

The bottleneck is silence. The verifier is calibrated and never false-positives on the fixed side, yet abstains on half of all judgements and misses most fixes. Optimising for precision alone would miss this. We argue abstention should be reported as a first-class outcome alongside precision and recall.

Attacks target abstention. Because the verifier already abstains when evidence is thin, an adversary does not need to convince it a vulnerability is safe — only to make it unsure. Defences that only suppress false positives leave the dominant path open.

Composition relocates trust. Grounding a claim in verifiable evidence is cheap and strictly helpful here. Requiring consensus with a weaker verifier inherits that verifier's blind spots. “Add another verifier” is not a reliability strategy; “make the existing verdict answer for itself” is.

A practical pitfall. A reasoning-model verifier silently degrades to “always abstain” when the token budget is consumed by reasoning tokens (`finish_reason=length`, empty content). Without the decomposition this is indistinguishable from a null result.

VII. THREATS TO VALIDITY

Scale. The scaled result is 21 security pairs; cross-family claims are on 5 pairs. Numbers are directional, not precise; we do not report confidence intervals for the small-sample comparisons. **Labels.** JavaVFC is a screening corpus; a companion audit [15] found only ~18% of it is a genuine security fix, so recall is a floor. **Input shape.** Each revision is an isolated file — a floor for tools that resolve types. **Non-determinism.** Reasoning models vary at temperature 0; we observed 38.1% and 28.6% detection for the same configuration. **Single family at scale.** Only `deepseek-flash` was run on the scaled corpus. **Contamination.** Public fixes may appear in pretraining data; a temporal or private holdout is not yet used. **Adversary.** Templates are fixed; one adaptive attacker was used with shallow adaptivity.

VIII. ETHICS AND RESPONSIBLE RELEASE

The adversarial comment generator is dual-use. This work is defensive: its purpose is to measure and expose a failure mode so that verifiers can be made robust. We do not release a turnkey attack service; the generator is a small, reproducible component of a measurement harness, and the paper reports defences (evidence gating, coverage reporting, abstention accounting) alongside the risk.

IX. REPRODUCIBILITY

The harness, pinned analyzers and rule sets, control set, and corpora are released. The control reproduction (12/18 vulnerable, 1/3 safe) and the Python-pair measurement run fully offline; 112 tests accompany the code. A verdict cache makes repeat runs deterministic and free.

X. CONCLUSION AND FUTURE WORK

LLM security verifiers track fixes far better than pattern matchers, are calibrated and precise when they commit, and are nonetheless *silent* on about half of their inputs — a weakness that adversarial comments amplify, and that composition with deterministic tools does not fix. Future work must (i) scale to hundreds of genuine fixes across at least two languages and three model families with confidence intervals, (ii) evaluate path-level candidates on multi-file revisions, and (iii) treat abstention and coverage as first-class metrics for security agents.

REFERENCES

[1] Y. Li et al., “Graph Is the Verifier: Agentic Reinforcement Learning for Interprocedural Vulnerability Detection,” arXiv:2607.26656, 2026.

[2] S. Vijay et al., “Antares: Foundation Models for Agentic Vulnerability Localization,” arXiv:2608.02407, 2026.

[3] S. Pan et al., “Answer Is Cheap, Show Me the Evidence! Augmenting Automated Vulnerability Assessment with Evidence,” arXiv:2608.25905 (ISSTA), 2026.

[4] N. Korda and G. Evron, “OpenAnt: LLM-Powered Vulnerability Discovery Through Code Decomposition, Adversarial Verification, and Dynamic Testing,” arXiv:2606.19149, 2026.

[5] Z. Wu and C. Nita-Rotaru, “ALIBI: Adaptive Agentic Attacks on LLM-Based Vulnerability Detectors via Adversarial Code Comments,” arXiv:2607.24964, 2026.

[6] Z. Sheng et al., “FuzzingBrain V2: A Multi-Agent LLM System for Automated Vulnerability Discovery and Reproduction,” arXiv:2605.21779, 2026.

[7] M. Sun and G. Meng, “DREA: Decoupled Reasoning and Exploration Agents for Repository-Level Vulnerability Detection,” arXiv:2607.13439 (Internetware), 2026.

[8] S. Yun et al., “CLEAR: Causal Context-Based Agentic Reasoning for Vulnerability Detection,” arXiv:2608.03134, 2026.

[9] T. Kim et al., “ATLANTIS: AI-driven Threat Localization, Analysis, and Triage Intelligence System,” arXiv:2509.14589, 2025.

[10] B. Zhang et al., “CoER: Defending against Adaptive Indirect Prompt Injection via Adversarial Co-Evolution and Refinement,” arXiv:2609.07529, 2026.

[11] F. Zhang et al., “To Answer or to Abstain: Mitigating Search-Agent Hallucinations via Abstention-Aware Reinforcement Learning,” arXiv:2607.10738, 2026.

[12] OWASP Foundation, “OWASP Benchmark Project,” 2026. [Online]. Available: https://owasp.org/www-project-benchmark/

[13] JavaVFC authors, “JavaVFC: A Corpus of Vulnerability-Fixing Commits for Java,” Zenodo record 13731781, 2026.

[14] “A Benchmark Score That Nothing Can Be Concluded From,” technical report and artifact (sastaudit), 2026. Introduces the four-way decomposition.

[15] W. Lin, T. Zhou, Z. Xie, and C. Hong, “Large Language Models at the Intersection of Software Engineering and Software Security: An Evidence-Centered Structured Survey and Research Agenda,” arXiv:2608.21107, 2026.

Rendering note: this PDF is generated from an HTML mirror of `paper/main.tex` for preview. The submission source is the IEEEtran LaTeX file; compile with `latexmk -pdf main.tex` where a TeX toolchain is available.